

PRAKTIKUM 1
FONDASI TEORETIS DAN KERANGKA KERJA DEVSECOPS

Disusun untuk Memenuhi Matakuliah Workshop DevOps

Dosen Pengampu : Bapak Dr. Ferry Astika Saputra, S.T., M.Sc.



Oleh :

Nama : Putra Yanuar Rasyid

NRP : 3126640058

Politeknik Elektronika Negeri Surabaya
Departemen Teknik Informatika dan Komputer
Program Studi Teknik Informatika

2026

Praktikum 1 - Menetapkan Baseline Laboratorium

Praktikum ini membuat direktori kerja tunggal dan merekam versi komponen. Rekaman versi diperlukan karena hasil scan dan perilaku tool dapat berubah. Jalankan pada host Linux/VM yang memang disiapkan untuk laboratorium.

```
mkdir -p ~/devsecops-lab/{app,policy,reports,sbom,keys}
```

```
zkhugger@zkhugger:~$ mkdir -p ~/devsecops-lab/{app,policy,reports,sbom,keys}
```

Analisis: Membuat folder utama lab beserta 5 sub-folder (app, policy, reports, sbom, keys) sekaligus dalam satu perintah

```
cd ~/devsecops-lab
```

```
zkhugger@zkhugger:~$ cd ~/devsecops-lab
```

Analisis: Berpindah masuk ke dalam direktori kerja lab

```
docker version
```

```
zkhugger@zkhugger:~/devsecops-lab$ docker version
Client: Docker Engine - Community
Version:      29.7.2
API version:  1.55
Go version:   go1.26.5
Git commit:   a7dcaa6
Built:        Wed Aug  5 18:28:40 2026
OS/Arch:      linux/amd64
Context:      default

Server: Docker Engine - Community
Engine:
Version:      29.7.2
API version:  1.55 (minimum version 1.40)
Go version:   go1.26.5
Git commit:   6a43e3d
Built:        Wed Aug  5 18:28:40 2026
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      v2.3.4
GitCommit:    db8809540e1a7a9da5d518876894933ff55692ab
runc:
Version:      1.4.3
GitCommit:    v1.4.3-0-gbb14dabe
docker-init:
Version:      0.19.0
GitCommit:    de40ad0
```

Analisis: Memeriksa versi dan status koneksi client ke Docker daemon

docker compose version

```
zkhugger@zkhugger:~/devsecops-lab$ docker compose version
Docker Compose version v5.5.0
```

Analisis: Memeriksa versi Docker Compose

git --version

```
zkhugger@zkhugger:~/devsecops-lab$ git --version
git version 2.53.0
```

Analisis: Memeriksa versi git

openssl version

```
zkhugger@zkhugger:~/devsecops-lab$ openssl version
OpenSSL 3.5.5 27 Jan 2026 (Library: OpenSSL 3.5.5 27 Jan 2026)
```

Analisis: Memeriksa versi OpenSSL

curl --version

```
zkhugger@zkhugger:~/devsecops-lab$ curl --version
curl 8.18.0 (x86_64-pc-linux-gnu) libcurl/8.18.0 OpenSSL/3.5.5 zlib/1.3.1 brotli/1.2.0 zstd/1.5.7 libidn2/2.3.8 libpsl/0
.21.2 libssh2/1.11.1 nghttp2/1.68.0 librtmp/2.3 mit-krb5/1.22.1 OpenLDAP/2.6.10
Release-Date: 2026-01-07, security patched: 8.18.0-lubuntu2.4
Protocols: dict file ftp ftps gopher gophers http https imap imaps ipfs ipns ldap ldaps mqtt pop3 pop3s rtsp scp sf
tp smb smbs smtp smtps telnet tftp ws wss
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL t
hreadsafe TLS-SRP UnixSockets zstd
```

Analisis: Memeriksa versi Curl

docker info --format '{{json .SecurityOptions}}'

```
zkhugger@zkhugger:~/devsecops-lab$ docker info --format '{{json .SecurityOptions}}'
["name=apparmor","name=seccomp,profile=builtin","name=cgroupns"]
```

Analisis: Mengambil dan menampilkan modul keamanan kernel host yang aktif di Docker

Verifikasi dan Skenario Pengujian

- Docker daemon dapat diakses oleh akun praktikum dan docker compose menggunakan plugin v2.

```
zkhugger@zkhugger:~/devsecops-lab$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
zkhugger@zkhugger:~/devsecops-lab$ docker compose version
Docker Compose version v5.5.0
```

Analisis: User non-root berhasil menjalankan perintah Docker tanpa sudo, dan CLI menggunakan plugin Compose v2

- Direktori reports, sbom, dan keys tidak dipublikasikan sebagai web root.

```
zkhugger@zkhugger:~/devsecops-lab$ ls -ld ~/devsecops-lab/{reports,sbom,keys}
drwxrwxr-x 2 zkhugger docker 4096 Aug 31 08:00 /home/zkhugger/devsecops-lab/keys
drwxrwxr-x 2 zkhugger docker 4096 Aug 31 08:00 /home/zkhugger/devsecops-lab/reports
drwxrwxr-x 2 zkhugger docker 4096 Aug 31 08:00 /home/zkhugger/devsecops-lab/sbom
```

Analisis: Folder sensitif berada di home user (~/devsecops-lab/), terpisah dari path web server publik sehingga aman dari akses langsung via browser/HTTP

- SecurityOptions menampilkan mekanisme host yang tersedia; hasil berbeda antardistribusi harus dicatat.

```
zkhugger@zkhugger:~/devsecops-lab$ docker info --format '{{json .SecurityOptions}}'
["name=apparmor","name=seccomp,profile=builtin","name=cgroupns"]
zkhugger@zkhugger:~/devsecops-lab$
```

Analisis: Menampilkan proteksi kernel bawaan seperti **AppArmor** (pembatasan akses/kapabilitas), **Seccomp** (filter syscall), dan **Cgroupns** (isolasi resource)

- Pembaca menulis satu paragraf threat statement: aset, aktor ancaman, jalur serangan, dan dampak.

Aset kritis mencakup private key di keys, laporan audit di reports, manifest dependensi di sbom, dan source code di app. **Aktor ancaman** berupa penyerang eksternal di jaringan yang mencari celah miskonfigurasi sistem. **Jalur serangan** terjadi jika folder sensitif tersebut tidak sengaja di-mount ke public web root atau diakses melalui celah *Local File Inclusion* pada aplikasi. **Dampak** serangannya meliputi kebocoran kunci rahasia, manipulasi rantai pasok software (*supply chain attack*), serta potensi pengambilalihan kontrol host server

Analisis Hasil

Lingkungan lab DevSecOps berhasil disiapkan dengan struktur direktori kerja terisolasi di path pengguna (~/devsecops-lab/) sehingga artefak sensitif (keys, reports, sbom) aman dari eksposur web root publik, didukung oleh Docker daemon yang dapat diakses non-root serta integrasi Docker Compose v2. Fitur keamanan host aktif melalui modul kernel Ubuntu berupa AppArmor, Seccomp, dan Cgroupns (berbeda dengan distro berbasis RHEL yang mengandalkan SELinux) untuk memitigasi ancaman aktor eksternal yang memanfaatkan jalur serangan salah konfigurasi volume mount atau celah Local File Inclusion, yang berpotensi menimbulkan dampak kebocoran aset private key, kompromi supply chain, hingga pengambilalihan kontrol server

Evaluasi dan Latihan Mandiri

1. Mengapa DevSecOps tidak dapat direduksi menjadi penambahan scanner pada pipeline?

DevSecOps memerlukan integrasi *culture*, desain *shift-left*, dan *automated remediation*. Penambahan *scanner* semata hanya memicu *alert fatigue* tanpa menjamin *vulnerability* ditindaklanjuti secara nyata

2. Evidence apa yang membedakan klaim kontrol dari kontrol yang benar-benar terverifikasi?

Bukan sekadar klaim dokumen, melainkan *technical artifacts* yang dapat diaudit, seperti *CI/CD build failure logs (automated quality gates)*, *digital signature* pada *container image* (Cosign), *SBOM manifest*, dan *audit trail* akses sistem

3. Bagaimana *shared responsibility* memengaruhi *ownership* risiko dan tindak lanjut temuan?

Membagi kepemilikan risiko secara eksplisit: *developer* bertanggung jawab atas *secure coding* dan *dependencies*, tim *ops* atas *runtime infrastructure*, dan tim *security* atas *governance policy*. Hal ini memperjelas *remediation owner* untuk setiap temuan teknis

Kesimpulan

Implementasi DevSecOps pada praktikum ini membuktikan bahwa keamanan sistem tidak hanya bergantung pada *tooling* atau *scanner*, melainkan pada arsitektur yang terisolasi dengan benar, kepatuhan *least privilege*, serta verifikasi berbasis *technical evidence*. Setup *environment* pada Ubuntu VM menunjukkan pentingnya isolasi direktori sensitif dari *public web root*, pemanfaatan modul keamanan kernel (*AppArmor* dan *Seccomp*), serta penerapan model *shared responsibility* agar setiap risiko dan celah teknis memiliki *remediation owner* yang jelas